

CSC11006 - CLOUD IMPLEMENTATIONS AND OPERATIONS PROJECT

Project 2: Building a Private Cloud Based on OpenStack (using DevStack deployment option)

Corrected Edition — with environment, hardware, and networking fixes applied

I. General information

- ID: PROJECT2
- Period: 3 weeks
- Deadline: (as announced in course schedule)
- Team: Group of 3 students

II. Outcome

By completing this project the students will understand:

- How DevStack implements OpenStack services.
- Neutron networking: private, DMZ, external, routers, and security groups.
- Instance lifecycle, floating IPs, SSH and network access.
- Persistent storage via Swift and Cinder, snapshotting volumes.
- Infrastructure-as-Code with Heat templates and cloud-init automation.

III. Describe

Objective: This project aims to provide hands-on experience in building and operating a private cloud using the OpenStack platform. Students will apply SysOps principles and networking knowledge to create, deploy, and orchestrate their first service, with an emphasis on leveraging the DevStack deployment option where appropriate.

Project Duration: 3 weeks.

Environment Setup (Prerequisite)

1. Provision an Ubuntu VM — 22.04 LTS, 16 GB+ RAM, 4+ vCPUs, and 60–80 GB disk.
2. Install DevStack:
 - Clone the devstack repo and check out a specific stable branch — do not leave this to chance. Pin the same branch across all team members, e.g.:

```
git clone https://opendev.org/openstack/devstack -b stable/2024.2
cd devstack
```

- Configure local.conf for your environment (single-node, all-in-one). At minimum, include:

```
[[local|localrc]]
ADMIN_PASSWORD=<Your_password>
DATABASE_PASSWORD=$ADMIN_PASSWORD
RABBIT_PASSWORD=$ADMIN_PASSWORD
SERVICE_PASSWORD=$ADMIN_PASSWORD
HOST_IP=<Your VM IP address>

# Heat is NOT installed by default - required for Module 5
<please fill your code to enable heat>
```

```
# Swift is NOT installed by default - required for Module 4
<please fill your code to enable swift>
```

3. Run `./stack.sh` to launch the all-in-one OpenStack environment. Expect 20–40 minutes with Heat and Swift both enabled.
4. Confirm the Horizon dashboard and openstack CLI work, and specifically confirm Heat and Swift are active before starting Module 1:

```
openstack orchestration service list    # should show heat-engine as up
openstack container list                # should return an empty list, not an error
```

Project Modules

Module 1: Getting Started with Horizon and Core Services (2 pt.)

Goal: Learn OpenStack services through the Horizon interface.

Steps:

- Log in to Horizon as admin or demo user.
- Explore Compute: Instances, Images, Key Pairs, Security Groups.
- Visit Network: Networks, Subnets, Routers.
- Open Orchestration (Heat) dashboard.
- Launch and terminate an instance via both Horizon and CLI.

Deliverable: Screenshots of each service dashboard and CLI commands used.

Module 2: Create Your First Network Topology (2 pt.)

Goal: Build tenant isolation with private, DMZ, and external networks.

Steps:

- Create a tenant-specific network "dmz-net" with a subnet.
- Create another internal network "private-net".
- Create a router, set its external gateway to the shared/public provider network, and add both dmz-net and private-net as router interfaces so each has a path out through the same router.
- Add Security Groups to allow certain traffic.
- Verify connectivity between networks.

Deliverable: Diagram of topology, list of commands, and visibility of network ports/subnets in Horizon.

Module 3: Deploying Your First Service (2 pt.)

Goal: Launch a simple "Hello World" web server instance reachable from the Internet.

Steps:

- Launch an instance attached to both DMZ and internal networks (i.e., with two NICs, one per network).

NOTE: A dual-homed instance (two NICs, potentially two gateways) commonly causes asymmetric routing inside the guest OS — traffic goes out one interface but replies try to return via the other, or the OS picks the wrong default route. Before troubleshooting connectivity, check the instance's routing table (`ip route`) and make sure only the DMZ-facing interface holds the default route; use static routes for the private-net side if needed.

- Assign a floating IP from the external network to the port on the DMZ-facing interface specifically — not the private-net port.

- Configure Security Group to allow SSH (tcp/22) and HTTP (tcp/80).
- SSH into the instance and install a minimal HTTP server (e.g., `python3 -m http.server 80`) or serve a "Hello World" HTML page.
- From your host or an external client, open the floating IP in a browser and confirm.

Deliverable: HTTP response screenshot, instance details, security group rules.

Module 4: Add Persistence to Your Services (2 pt.)

Goal: Use object storage (Swift) and block storage (Cinder); snapshot and recovery.

Steps:

- Create a Swift container (e.g., "tetris-src").
- Upload source code (e.g., a tetris example) to the container.
- Launch or reuse an existing instance; create a Cinder volume and attach it to the instance.

NOTE: Confirm the instance can actually reach the Swift API endpoint before this step. Depending on how Module 2's topology was built, private-net alone may not route to Swift (typically reached via the controller's API/management network). If the download from Swift fails with a connection error rather than an authentication error, check routing/floating IP access from the instance first — this is a common gap between the Module 2 network design and the Module 4 storage step.

- Mount the volume and deploy the uploaded code from Swift onto it.
- Run the application (e.g., server code).
- Take a snapshot of the volume.
- Detach the volume and launch a new volume from the snapshot. Reattach it to a new instance and verify data persistence.

Deliverable: Logs/screenshots of Swift container, volume, snapshot, and re-attached instance showing the code.

Module 5: Orchestrate Your Service (2 pt.)

Goal: Automate deployment via a Heat template including network, instance, volumes, and userdata/cloud-init.

Steps:

- Write a Heat template (service.yaml) that:
 - Defines networks, router, security groups,
 - Launches the web server VM,
 - Attaches a Cinder volume,
 - Embeds a cloud-init script to download code from a provided URL and start the service.
- Includes userdata to run a post-configuration script.
- Deploys the stack via Horizon or CLI (openstack stack create).
- Verifies stack creation, infrastructure, and service accessibility.

Deliverable: The template file, stack outputs, and evidence that the web service is running correctly.

NOTE: Heat Orchestration Template (HOT) syntax and resource-dependency modeling are typically a full course unit on their own. If this is most students' first exposure to Heat, consider providing a minimal starter service.yaml (a single network + single instance, no volume) that teams extend, rather than expecting a template to be authored entirely from scratch under the module's time budget.

Project Structure

Submit a .zip file. Organize files like:

```
/StudentID1-StudentID2-StudentID3
  Report.pdf      -> describes the project process, research & knowledge
  module2_diagram.png
  module5_service.yaml
  demo.txt        -> link to video demo
```

- Include instructions for reproducing each step.
- Provide context: Ubuntu version (including RAM, vCPU, disk storage), DevStack branch used.

Tips & Best Practices

- Use the openstack CLI in parallel with Horizon to reinforce understanding.
- Clearly document Security Group rules, port mappings, and floating IP settings.
- In Heat, parameterize values for reuse.
- Use separate projects (tenants) to compartmentalize lab activity.
- Verify Heat and Swift are active immediately after ./stack.sh completes, before starting Module 1 — don't discover a missing service mid-project.
- Optional: automate cleanup with openstack stack delete or a script.